```
                    if: ["market.retail?", :desktop?],
                    unless: Proc.new { |c| c.trackpad.present? }
end
```

The validation only runs when all the `:if` conditions and none of the `:unless` conditions are evaluated to `true`.

6 Performing Custom Validations

When the built-in validation helpers are not enough for your needs, you can write your own validators or validation methods as you prefer.

6.1 Custom Validators

Custom validators are classes that inherit from `ActiveModel::Validator`. These classes must implement the `validate` method which takes a record as an argument and performs the validation on it. The custom validator is called using the `validates_with` method.

```
class MyValidator < ActiveModel::Validator
  def validate(record)
    unless record.name.starts_with? 'X'
      record.errors[:name] << 'Need a name starting with X please!'
    end
  end
end

class Person
  include ActiveModel::Validations
  validates_with MyValidator
end
```

The easiest way to add custom validators for validating individual attributes is with the convenient `ActiveModel::EachValidator`. In this case, the custom validator class must implement a `validate_each` method which takes three arguments: record, attribute, and value. These correspond to the instance, the attribute to be validated, and the value of the attribute in the passed instance.

```
class EmailValidator < ActiveModel::EachValidator
  def validate_each(record, attribute, value)
    unless value =~ /\A([^@\s]+)@((?:[-a-z0-9]+\.)+[a-z]{2,})\z/i
      record.errors[attribute] << (options[:message] || "is not an
email")
    end
  end
end

class Person < ActiveRecord::Base
  validates :email, presence: true, email: true
end
```

As shown in the example, you can also combine standard validations with your own custom validators.

6.2 Custom Methods

You can also create methods that verify the state of your models and add messages to the `errors`collection when they are invalid. You must then register these methods by using the `validate` class method, passing in the symbols for the validation methods' names.

You can pass more than one symbol for each class method and the respective validations will be run in the same order as they were registered.

```
class Invoice < ActiveRecord::Base
  validate :expiration_date_cannot_be_in_the_past,
    :discount_cannot_be_greater_than_total_value
```